

CISC 322/326
Assignment 3: GNUstep Architectural Enhancement
April 4, 2025

GNU16 Design Team
Group 16

Group Leader: Luc Robitaille - 21lar18@queensu.ca
Presenter 1: Enqi Liang - 21el32@queensu.ca
Presenter 2: Rodrigo Del Aguila Velarde - 20rdav@queensu.ca
Vlad Kukov - 21vk29@queensu.ca
Lucas Ferber - 19ljf1@queensu.ca
Robert He - 20rlth@queensu.ca

Table of Contents

1. Abstract
2. Introduction
3. Proposed Enhancement
 - 3.1. Value and Benefit of Feature
 - 3.2. Changes to Architecture
 - 3.3. Interactions With Other Features
 - 3.4. Planned Tests
 - 3.5. Risk Analysis
4. Software Architecture Analysis Method (SAAM)
5. Use Cases
6. Lessons Learned
7. Data Dictionary
8. Conclusion
9. References

1. Abstract

This report conceptualizes and presents a potential enhancement to the GNUstep framework. To begin with, the proposed enhancement is discussed in detail. This section starts with explanations of the value and benefits that would come from our proposed enhancement. Then the architectural changes that will need to be made in GNUstep for our enhancement are presented. Next, the new interactions with other features created from the implementation of our enhancement are showcased. Following this section, the report lays out the planned testing for the enhancement, and then a risk analysis is conducted to showcase the potential risks of our proposed change. After the in-depth discussion about our proposed enhancement, the report will then provide an SEI SAAM architectural analysis of our proposed enhancement. This analysis recognizes the major stakeholders of the enhancement, while also identifying the most important non-functional requirements of the enhancement. The second half of the architectural analysis showcases two ways that the proposed enhancement could be implemented within GNUstep, with explanations on how they impact the non-functional requirements and stakeholders. This architectural analysis then presents the best overall implementation of our enhancement, with the reasoning coming from the earlier steps of the analysis. Next, this report provides two distinct use cases connected to our enhancement, with corresponding sequence diagrams. Finally, this report discusses our lesson learned during this process, and provides our final conclusions on our proposed enhancement.

2. Introduction

When creating a desktop application, one of the most important aspects a developer has to consider are the visuals. The style and overall look of an application heavily affects the common consumer's decision about what applications they want to use. This is where we identified a major problem with the GNUstep framework. Currently, the theming options for GNUstep are based on the NeXTstep theming options from the 1990's. This makes the base theming options look extremely outdated. There are also theming options to match the aesthetic of popular operating systems, like Windows for example [1]. The only issue with this theme, is that the Windows theming is based on the outdated aesthetics of Windows XP and Windows Vista (See Figure 1). It's important to clarify that the theming options for how an application looks is larger than you would think. The theming applies to buttons, drop-down menus, text-fonts, and check-boxes to name a few [1]. All of these aspects are needed in some way when creating a desktop application. That is why we decided our proposed new feature would actually be an enhancement on the GNUstep theming packages, focusing on bringing these outdated theming packages up to modern standards.

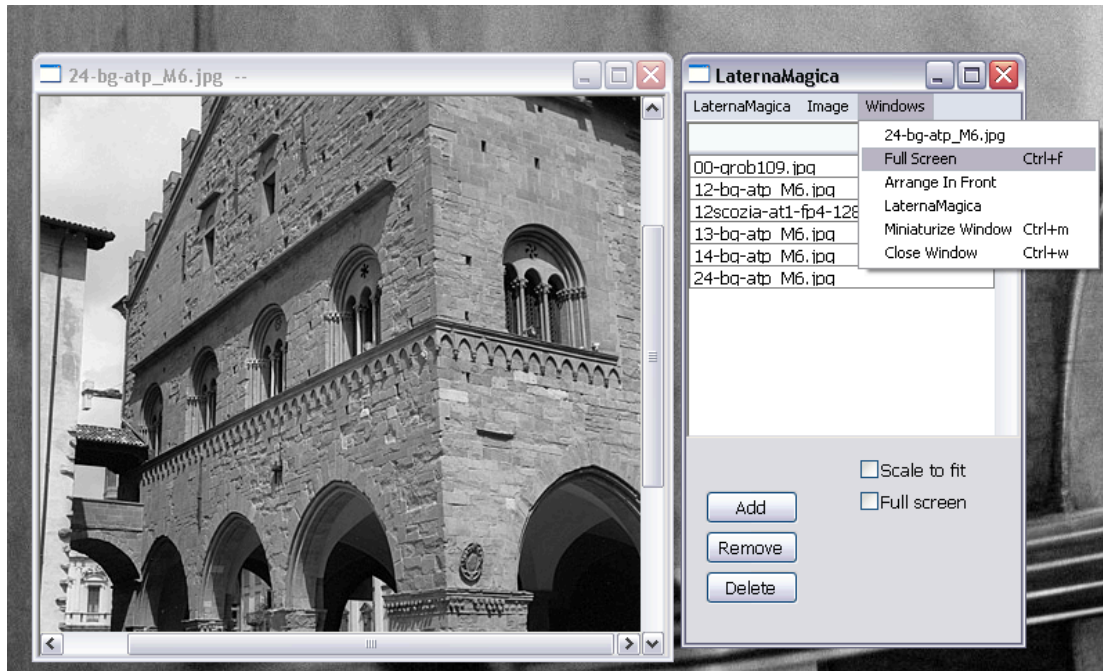


Figure 1: GNUstep Application Created With the Windows (WinUXTheme) in GNUstep [1]

3. Proposed Enhancement

Introduce a CSS-like theming system, high-DPI scaling, and system UI integration while enabling customizable widget styles, adaptive padding, and modern UI components for an enhanced user experience.

Our proposed enhancement is to update and modernize the theming options for desktop applications made with the GNUstep framework.

3.1 Value and Benefit of Feature

The value of modernizing theming options for applications, is that users of the GNUstep framework would be able to make desktop applications that match the aesthetics of modern computers. The main benefit of updating GNUstep's themes is that developers using the GNUstep framework would have the ability to create desktop applications with modern standards visually. This results in developers no longer feeling restricted on the potential of their desktop programs while using the GNUstep framework. This not only unlocks the potential for GNUstep veterans to be able to make programs more accessible on modern desktops. But also opens the door for more new developers to learn and use the GNUstep framework, as it would now easily support modern aesthetics within desktop application development.

3.2 Changes to Architecture

The `libs-gui` will be the library most affected by our proposed enhancement. Currently, the `GNUstep` theming packages are connected to the `libs-gui` library, as theming is a visual aspect of applications, and would need to be handled mostly by the front-end of the application. Most of the changes would be simply updating and adding new files corresponding to the visual assets of the pre-existing theming package. Depending on the scope of the introduced modern theming, more features for theme customization could be introduced. These additional features would require changes to the `libs-gui` and `libs-back` library. For the `libs-gui` library, changes would have to be made to ensure that the additional theming customization features would render correctly through the library handling the front-end. The `libs-back` library would also need changes for the additional features as new code would have to be introduced to handle the functionality of the potential theming customization features. There would also be more changes to the `libs-back` library, as some files might need to be changed to be able to handle the new assets/files created for the new theming capabilities.

3.3 Interactions with Other Features

The proposed enhancements would interact heavily with the existing features of `GNUstep`. Primarily with the `GNUstep-gui` and `GNUstep-back` libraries, as this is where most of the rendering and UI organization take place. The enhancement would require extending many of the classes and functions in these libraries in order to apply the CSS-like style rules to various UI elements like widgets, borders, backgrounds and margins. Updates to many of the layout management classes would need to be implemented in order to dynamically adjust based on theme rules. The `GNUstep` theming system would also need to be heavily overhauled or replaced in order to provide the features implemented by a CSS-like theming convention. In the `GNUstep-back` library, many classes that are responsible for rendering and scaling would need to be extended in order to ensure that fonts are rendered correctly and UI elements are aligned properly at different screen resolutions and scaling levels. These enhancements would also interact heavily with the `Application kit` and the various applications that rely on `GNUstep`. Additional classes and layers would need to be implemented in order to maintain backwards compatibility with applications that do not support CSS based styling.

3.4 Planned Tests

There are several tests that need to be conducted to determine the impact of the proposed enhancements to ensure compatibility, performance and user experience improvements while maintaining system stability. The first test would need to ensure that the CSS-like theming system integrates with existing UI components and that it

does not break any existing applications. This could be done by testing backwards compatibility with older themes and non CSS based styles. For the high DPI scaling feature, the performance and effectiveness would need to be evaluated across different screen resolutions and UI elements. The responsiveness and performance at different scaling levels would need to be measured and how different themes and widget styles adapt to different DPI settings would need to be verified. Different themes would also need to be checked for blurry text or misaligned elements. The customizable widget styles would need to be tested to ensure seamless compatibility and that it does not break layouts and accessibility. Various widget styles would need to be applied and tested to see how they respond to different screen sizes and DPI settings while verifying correct widget alignment with both standard and modern UI components.

3.5 Risk Analysis

An enhancement of this size and complexity comes with many risks that may jeopardize the integrity of the GNUstep system. Implementing this enhancement risks creating weaknesses in the security of GNUstep due to it requiring major changes to many of the subsystems. If not implemented properly, these changes could create areas where users can exploit the system in ways not intended by the developer. This enhancement also potentially creates problems with maintainability. All of the subsystems, now with many more classes and functions, need to be maintained and the added complexity might be too much for GNUstep's exclusively volunteer workforce. Any bugs potentially created with such an enhancement would also need to be fixed and due to GNUstep's extremely slow development cycle these bugs could take years to fix, potentially alienating what few users it has left. This enhancement would be very large and would be very intensive to implement and as such simply undertaking this project is a risk in itself. GNUstep relies solely on volunteer software developers to work on changes and improvements for free in their spare time and as such their workforce is very small and sporadic. An enhancement of this size may be too much for the small developer team to accomplish and as such there is a risk of this enhancement never getting completed.

4. Software Architecture Analysis Method

The main beneficiaries of improved UI development tools would obviously be developers, who will be able to create more modern looking applications for either their private, or public use. For these developers, it is of critical importance that they have the tools to keep their applications looking up to date, and not something out of the early 2000s. It is simply not practical to expect users to comfortably use tools designed for a device 10 years ago. If the additional display features and themes supported by modern

devices are not supported by GNUstep, developers will be unable to create modern looking applications, and any app they do create will look out of place. More development tools give developers more options on editing the appearance of their application. Allowing them to edit the UI to their own preference.

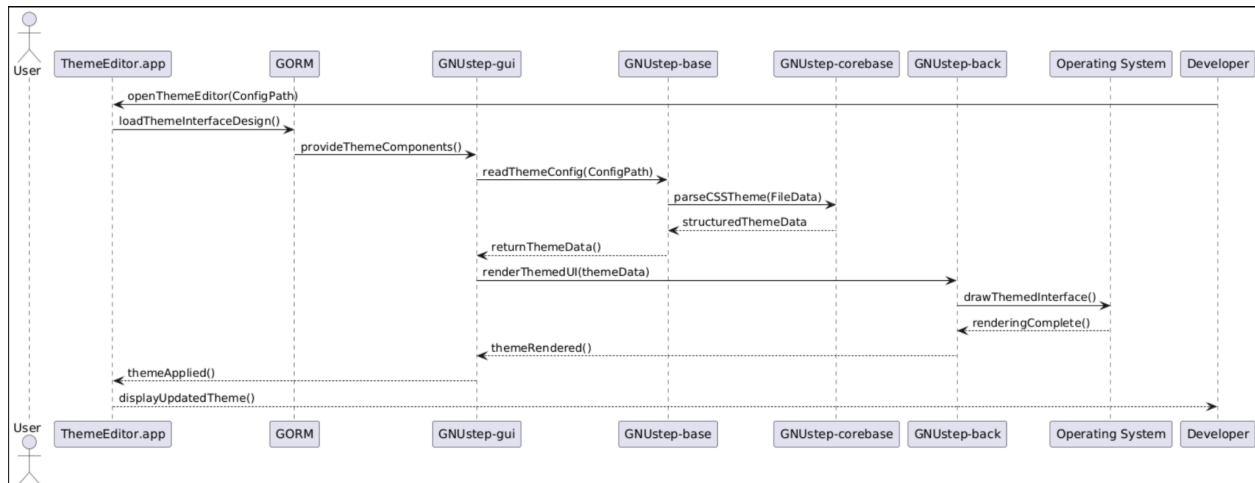
Two proposed methods for enhancing the UI include CSS-based theming Support, and system UI integration. The potential benefits of either of two methods revolve around customizability, and accessibility. An CSS-like system gives developers customizability, since these CSS configuration files would be user created. The files are custom created by the developers, so they have greater autonomy in controlling the appearance of their application. A modernised UI can also be created with CSS, since developers will be able to design their own themes, rather than being restricted to the ones currently offered. System UI integration on the other hand, would automatically adapt applications to system themes in the operating OS. Since the appearance is automatically adapted, a modern UI appearance is more accessible, and is easier to configure for the developer. But there is a loss of customizability, as the developer is restricted to using these prebuilt themes.

Overall, CSS-like theming support seems like the better choice, even if it's harder to use. It would allow the developer to create their own modern looking UI, while giving them customisable options that UI integration would not be able to offer.

5. Use Cases

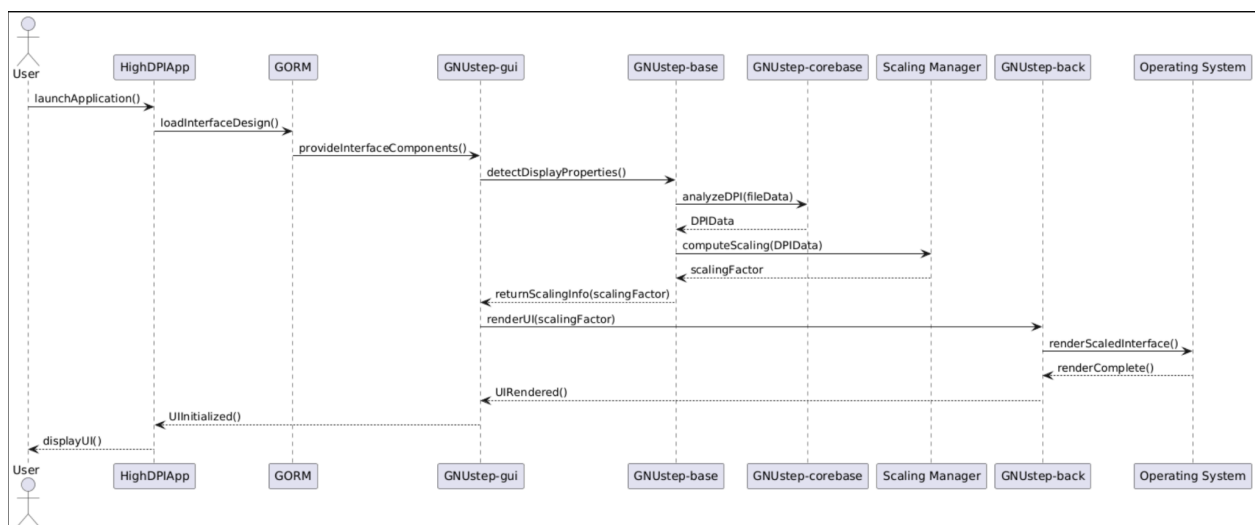
Use Case 1:

A user can customize the new CSS-like theming options to update the visual appearance of a GNUstep desktop application. In the use case, the user starts by opening the ThemeEditor.app with the path to the theme configuration file. The editor then loads the theme interface design by calling GORM, which will send the necessary theme components to the GNUstep-gui. Next, GNUstep-gui reads the theme configuration file by interacting with GNUstep-base. GNUstep-base then calls GNUstep-corebase to parse the CSS-like theme file data, and the structured theme data is returned back up. With theme data ready, GNUstep-gui sends it to GNUstep-back to render the updated themed UI. The Operating System renders the new interface, confirms that rendering is complete and the ThemeEditor.app will display the updated theme to the Developer.



Use Case 2:

When an application is launched on a high-DPI display, the GNUstep framework automatically detects the environment, using the appropriate scaling factors, adjusting the UI components. In this use case, when the User launches the HighDPIApp, the application begins to load the interface design through GORM, which provides the interface components using GNUstep-gui. GNUstep-gui then asks GNUstep-base to detect the display properties. GNUstep-base uses GNUstep-corebase to analyze the DPI data from the file. GNUstep-base gets the data, GNUstep-base sends it to the Scaling Manager to compute the scaling factor. The scaling factor is returned to GNUstep-base and then passed along to GNUstep-gui. GNUstep-gui instructs GNUstep-back to render the UI using the computed scaling factor. The Operating System renders the scaled interface and confirms the completion of rendering. In the last step the HighDPIApp displays the properly scaled user interface to the User.



6. Lessons Learned

There were several lessons learned in our exploration of possible GNUStep enhancements. First of all, we discovered the importance of in-depth system understanding in the process of finding and proposing an implementation of an improvement in a large system. The process of determining a useful enhancement showed us how our prior knowledge of the system's limitations through our research can help provide an insightful improvement. Through our last two deliverables we observed how dated the system was and that allowed us to understand that the system needs a modernization. Additionally, our prior experience with the model's conceptual and concrete architecture allowed us to create a sophisticated implementation analysis. Since we knew the interactions of different parts of the GNUStep system, we were able to make a rough plan for the modification in significantly more detail than is possible with basic surface level knowledge.

Another lesson learned is the general process of modifying an existing system. This is a useful skill which can be applied in many contexts including jobs and personal projects. As we progressed through the assignment we learned how much more intricate the procedure of adding an improvement to a large codebase is than simply conceptualizing the idea and beginning programming. We found out that a good modification should consider the convoluted interactions between different components as well as any potential changes to the NFRs and the impacts on stakeholders. We also discovered that a good practice is to take into consideration multiple alternatives before settling a specific implementation.

7. Conclusion

In summary, the proposed enhancement to the GNUstep framework aims to modernize the outdated theming options by introducing a CSS-like theming system, high-DPI scaling, and improved system UI integration. This enhancement is designed to offer huge benefits to developers by enabling them to create applications that not only meet modern visual standards but also provide improved accessibility and usability on various screen resolutions. The architectural changes which primarily affect the libs-gui and libs-back libraries will ensure that the new features are properly integrated with existing systems at the same time maintaining backwards compatibility. The comprehensive testing strategy is crucial to verifying that the enhanced theming system works reliably with current applications, providing a more modern user interface without regressions. The enhancement also presents notable risks, including potential security vulnerabilities, maintainability challenges, and the reliance on a limited volunteer workforce, but the benefits of a modern, flexible, and developer-friendly framework are clear. Furthermore, our exploration of GNUStep enhancements revealed that deep

system understanding and careful planning are essential when modernizing a large codebase. Recognizing the system's limitations and intricacies from complex component interactions to potential impacts on non-functional requirements has emphasized the importance of research and evaluating multiple implementation alternatives before finalizing an improvement. In the end, the proposed enhancement not only addresses the need for modern theming options inside GNUMstep but also states a more reliable foundation for future improvements, which will ensure that the framework remains usable and modernized in the future.

8. Data Dictionary

CSS-like theming - A styling mechanism inspired by CSS using configuration files to improve customization and flexibility of the UI

High-DPI scaling - The process of a system adjusting size of UI elements such as text and icons to match screens with higher pixel density to improve clarity

System UI integration - Adaptation of UI to match the host operating system to provide a simpler implementation of a modern look

Widgets - Interactive components of a UI such as buttons and checkboxes

Adaptive padding - Automatic adjustment of spacing around UI elements based on display changes such as layout or resolution

Non-functional requirements - System behaviour attributes such as performance and security which evaluate the quality.

SAAM (Software Architecture Analysis Method) - Evaluation of a system architecture which focuses on an architecture's impact on stakeholders and NFRs

Rendering - The process of generating visual elements from a set of instructions

Backwards Compatibility - Compatibility of newer versions of a system with older versions for simple integration of new updates

9. References

[1] "Themes," GNUMstep, <https://www.gnumstep.org/experience/Themes.html>